

pa01

Antonio Lazzaro; MoWe 4-5:15

September 2026

1 Heuristic Design

Let B be the set of incorrect letters left in the current word, and $h(x)$ be the heuristic value of a set of letters.

1.1 Steps

$h(B) = \text{length}(B)$. The remaining cost must be at least one for each incorrect letter, since each of those letters must be changed at least once in order to change them to the correct letter, maybe more if it needs to be changed twice to take a detour to stay in the realm of real words, but it can't be less since we can only change one letter each step.

1.2 Scrabble

Let $s(x)$ be the Scrabble value for a letter x . Then, $h(B) = \sum_{i=1}^{\text{length}(B)} s(g_i)$. If g_i is the goal word counterpart of b_i . This means that the heuristic will be equal to the sum of the Scrabble values of the letters in the goal word that aren't already in the current word. Those letters must still be inserted, so their values will be included in the remaining cost. In the best-case scenario, only those letters will be inserted, so only their Scrabble values will be included in the remaining cost, so this heuristic can't be greater than the remaining cost.

1.3 Frequency

Let f be the frequency value of the goal word. Then, $h(B) = \text{length}(B) + f$. Since each step has a cost of 1 in addition to the dictionary value of the resulting word, then we know that the dictionary value for the goal word must be included in the remaining cost, and that 1 will be added each time we change a letter, so the number of incorrect letters must also be included in the remaining cost.

2 Heuristic Effectiveness

Word Pair	Cost measure	UCS cost	UCS nodes	A* cost	A* nodes	Reduction
Why, Ask	Steps	5	773	5	16	97.93%
Why, Ask	Scrabble	9	756	9	13	98.28%
Why, Ask	Frequency	38.86	808	38.86	755	6.56%
Bear, Duck	Steps	4	691	4	6	99.13%
Bear, Duck	Scrabble	11	2952	11	6	99.80%
Bear, Duck	Frequency	38.24	1084	38.24	720	33.58%

Table 1: UCS vs A* Performance

Both algorithms returned the same optimal cost every time. A* produced the largest proportional reduction in the scrabble measure for both word pairs. The reduction for scrabble was only slightly higher than steps, but they were both very high due to the fact that changes in individual letters affect the cost directly, which made them easier to account for in the heuristic. The heuristic was least informative for the frequency measure in both pairs, since the cost is measured from the whole word rather than individual letters, so it's hard to base the heuristic off changes to the letters.

3 Reversibility

3.1 Steps

The optimal cost works both ways for this measurement. The same path always has the same number of steps no matter which end you start at, so if the cost is purely based off the steps, then it will remain the same despite the orientation of the path.

3.2 Scrabble

No, the optimal cost could be different, since the values of the initial letters aren't included, then the orientation dictates which letter's values will be included in the cost. For example, d has a lesser value than j, so going from jog to dog will have a lower cost than going from dog to jog.

3.3 Frequency

Like scrabble, the dictionary value for the initial word isn't included in the cost. Thus, due to the same logic, switching the orientation switches what values are included. For example, going from bear to duck costs 38.24, while going from duck to bear costs 36.79.

4 Heuristic Audit

4.1 AI Model and proposed heuristic

The AI model used was Kimi Instant with high thinking effort. The exact prompt was:

"In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word costs $1 + \text{rarity}()$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic h_0 is the number of positions in which state differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By "more informative," we mean that the proposed heuristic should be at least as large as 0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost."

4.2 Proposed Heuristic

It proposes the heuristic

$$h_r = \begin{cases} 0, & n = g \\ d(n) + r(g) + (d(n) - 1)m, & n \neq g \end{cases}$$

It also defines $D(n)$ to be the Hamming distance n to the goal g (in other words the number of incorrect letters in the current word), and $r(w)$ to be the rarity of word w . As well as

$$m = \min_{w \in D, w \neq g} (r(w)).$$

4.3 Heuristic Admissibility

Let there be x incorrect letters in the current word. Then, $d(n)=x$. Now suppose the best case scenario, where no detours need to be taken to get to the goal word (i.e., only x substitutions need to be made). Then the cost C to get to the goal word is

$$C = \sum_{i=1}^x (1 + r(w_i))$$

if w_i is the current word after the i th substitution. Then, we know that

$$C = x + \sum_{i=1}^x r(w_i)$$

through basic algebra and that

$$C = x + r(g) + \sum_{i=1}^{x-1} r(w_i),$$

since we know the last word in the sequence will be the goal word. Since the heuristic must be a lower bound of the remaining cost, the following inequality must be true,

$$h(n) \leq C \implies d(n) + r(g) + (d(n) - 1)m \leq x + r(g) + \sum_{i=1}^{x-1} r(w_i).$$

We've already established that $d(n)=x$, so we can substitute into the inequality such that

$$x + r(g) + (x - 1)m \leq x + r(g) + \sum_{i=1}^{x-1} r(w_i).$$

Then the inequality remains true after subtracting $x+r(g)$ from both sides leaving

$$(x - 1)m \leq \sum_{i=1}^{x-1} r(w_i).$$

Finally, we know that $\forall w \in D, m \leq w$, by the definition of m . Hence, in the best case (which was supposed in the setup), all $r(w_i)$ in the summation are equal to m , which would make $(x - 1)m = \sum_{i=1}^{x-1} r(w_i)$ true, which implies the inequality is true. Therefore in the best case where the lowest possible cost remains, the heuristic is still not greater than the actual cost. We also know that it's greater than zero (except when the current word is the goal word), since each term is positive, which makes their sum positive.

4.4 Testing Proposed Heuristic

The percentage of reduction of expanded nodes for the default heuristic was 7% for (why, ask) and 33% for (bear, duck), while the reduction for the proposed heuristic was 22% for (why, ask) and 72% for (bear, duck). In both cases the proposed heuristic was significantly larger than the default, but still nowhere close to the heuristics for the other measurement methods. The optimal costs also concurred with one another. This is strong evidence supporting the use of the proposed heuristic.

5 Acknowledgments

As stated in the 4th section I used Kimi's Instant model for the heuristic prompt. I set the thinking parameter to high in hopes to receive the most coherent output. I also lightly used the VS Code integrated model of Microsoft Copilot for debugging and auto-correcting my code as I was writing it. Any corrections made by Copilot were tested through proofreading and running the functions they affected and debugging any issues that were found. The code skeleton was obviously provided in the assignment's folder on blackboard.